Introduction to Quality Metrics - 3

Lecture # 43

Function Points - 1

- Historically, lines of code count have been used to measure software size
- LOC count is only one of the operational definitions of size and due to lack of standardization in LOC counting and the resulting variations in actual practices, alternative measures were investigated
- One such measure is function point

Function Points - 2

- Increasingly function points are gaining in popularity, based on the availability of industry data
- The value of using function points is in the consistency of the metric
- If we use lines of code to compare ourselves to other organizations we face the problems associated with the inconsistencies in counting lines of code

Function Points - 3

- Differences in language complexity levels and inconsistency in counting rules quickly lead us to conclude that line of code counting, even within an organization, can be problematic and ineffective
- This is not the case with function points

Function Points - 4

- Function points also serve many different measurement types
- Again, lines of code measure the value of the rate of delivery during the coding phase
- Function points measure the value of the entire deliverable from a productivity perspective, a quality perspective, and a cost perspective

Common Industry Software Measures

Type	Metrics	Measure
Productivity	Function Points / Effort	Function points per person month
Responsiveness	Function Points / Duration	Function points per calendar month
Quality	Defects / Function Points	Defects per Function points
Business	Costs / Function Points	Cost per Function point

Function Points - 5

- A function point can be defined as a collection of executable statements that performs a certain task, together with declarations of the formal parameters and local variables manipulated by those statements
- The function points were proposed by Albrecht and his colleagues at IBM in mid-1970s

Function Points - 6

- It addresses some of the problems associated with LOC counts in size and productivity measures, especially the differences in LOC counts due to different levels of languages used
- It is a weighted total of five major components that comprise an application

Function Points - 7

- These five components are
  - Number of external inputs
  - Number of external outputs
  - Number of logical internal files
  - Number of external interface files
  - Number of external inquiries

- Example of a simple spell checker

External Inputs

- Those items provided by the user that describe distinct application-oriented data (such as file names and menu selections). These items do not include inquiries, which are counted separately
- For example, *document filename*, *personal dictionary-name*

External Outputs

- Those items provided to the user that generate distinct application-oriented data (such as reports and messages, rather than the individual components of these)
- For example, *misspelled word report*, *number-of-words-processed message*, *number-of-errors-so-far message*

External Inquiries

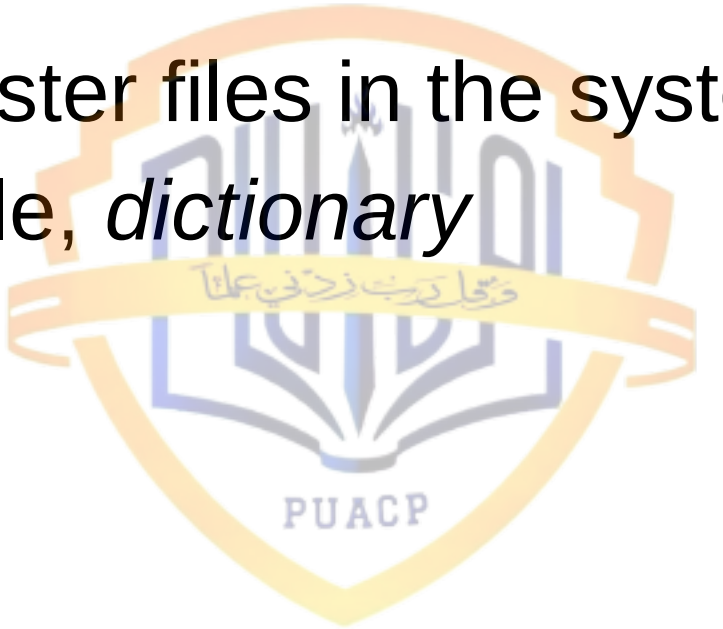
- Interactive inputs requiring a response
- For example, *words processed*, *errors so far*

External Files

- Machine-readable interfaces to other systems
- For example, *document file*, *personal dictionary*

Internal Files

- Logical master files in the system
- For example, *dictionary*



Function Points - 8

- Each item/component is assigned a subjective “complexity” rating on a three point ordinal scale: “simple”, “average”, or “complex”
- Then a weight is assigned to the item according to the following table

FP Complexity Weights

Item	Simple	Weighting Factor Average	Complex
External Inputs	3	4	6
External Outputs	4	5	7
External inquiries	3	4	6
External files	7	10	15
Internal files	5	7	10

Function Points - 9

- The complexity classification of each component is based on a set of standards that define complexity in terms of objective guidelines
- For instance, for the external output component, if the number of data element types is 20 or more and the number of file types referenced is two or more, then complexity is high
- If the number of data element types is 5 or fewer and the number of file types referenced is two to three

Function Points - 10

- With the weighting factors, the first step is to calculate the Unadjusted function counts (UFC) based on the formula, which is obtained by the by multiplying the weighting factors with the corresponding components, and adding them up
- There are fifteen (15) different varieties of items/components – in theory

Unadjusted Function Counts

- UFC = sum((Number of items of variety i) * weight(i))
  - For i= 1..15

Function Points - 11

- We calculate an adjusted function point count, FP, by multiplying UFC by a technical complexity factor, TCF
- This technical involves the 14 contributing factors

Components of the Technical Complexity Factor

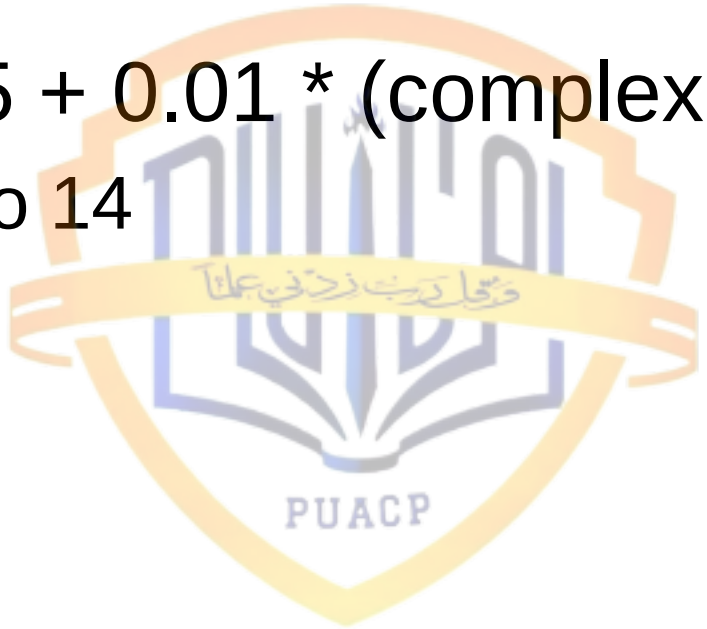
Reliable back-up and recovery	Data communications
Distributed functions	Performance
Heavily used configuration	Online data entry
Operational ease	Online update
Complex interface	Complex processing
Reusability	Installation ease
Multiple sites	Facilitate change

Function Points - 12

- The scores (ranging from 0 to 5) for these characteristics are then summed, based on the following formula, to form the technical complexity factor (TCF)

Technical Complexity Factor

- TCF = 0.65 + 0.01 * (complexity of item i)
  - For i = 1 to 14



Function Points - 13

- Finally, the number of function points is obtained by multiplying unadjusted function counts and the value adjustment factor:

FP = UFC * TCF

- Over the years, the function point metric has gained acceptance as a key productivity measure in the application world
- In 1986 the IFPUG was established. The IFPUG counting practices committee is the *de facto* standards organization for function point counting methods

Problems with Function Points - 1

- Problems with subjectivity in the technology factor
- Problems with double-counting
- Problems with accuracy of TCF
- Problems with early life-cycle use
- Problems with changing requirements

Problems with Function Points - 2

- Problems with methodology dependence
- Problems with application domain
- Problems with subjective weighting

Complexity

Say few words on complexity

Complexity - 1

- Ideally, we would like the complexity of the solution to be no greater than the complexity of the problem, but that is not always the case

Complexity - 2

- The complexity of a problem as the amount of resources required for an optimal solution to the problem
- The complexity of a solution can be regarded in terms of the resources needed to implement a particular solution

Complexity - 3

- Time complexity
  - Where the resource is computer time
- Space complexity
  - Where the resource is computer memory

Complexity - 4

- Problem complexity
- Algorithmic complexity
- Structural complexity
- Cognitive complexity

Problem Complexity

- Measures the complexity of the underlying problem
- This is also known as computational complexity by computer scientists)

Algorithmic Complexity

- Reflects the complexity of the algorithm implemented to solve the problem; in some sense, this type of complexity measures the efficiency of the software
- We use Big-O notation to determine the complexity of algorithms

Structural Complexity

- Measures the structure of the software used to implement the algorithm
- For example, we look at control flow structure, hierarchical structure, and modular structure to extract this type of measure

Structural Measures - 1

- All other things being equal, we would like to assume that a large module takes longer to specify, design, code, and test than a small one. But experience shows us that such an assumption is not valid; the structure of the product plays a part, not only in requiring development effort but also in how the product is maintained

Structural Measures - 2

- We must investigate characteristics of product structure, and determine how they affect the outcomes we seek
- Structure has three parts
  - Control-flow structure
  - Data-flow structure
  - Data structure

Control-Flow Structure

- The control-flow addresses the sequence in which instructions are executed in a program. This aspect of structure reflects the iterative and looping nature of programs. Whereas size counts an instruction just once, control flow makes more visible the fact an instruction may be executed many times as the program is actually run

Data-Flow Structure

- Data flow follows the trail of a data item as it is created or handled by a program. Many times, the transactions applied to data are more complex than the instructions that implement them; data-flow measures depict the behavior of the data as it interacts with the program

Data Structure

- Data structure is the organization of the data itself, independent of the program. When data elements are arranged as lists, queues, stacks, or other well-defined structure, the algorithms for creating, modifying, or deleting them are more likely to be well-defined, too. So the structure of the data tells us a great deal about the difficulty involved in writing programs to handle the data, and in defining test cases for verifying that the programs are correct
- Sometimes a program is complex due to a complex data structure rather than complex control or data flow

Control-Flow Structure - 1

- The control flow measures are usually modeled with directed graphs, where each node (or point) corresponds to a program statement, and each arc (or directed edge) indicates the flow of control from one statement to another
- These directed graphs are called control-flow graphs or flowgraphs

Control-Flow Structure - 2

- In-degree (Fan-in)
  - A count of the number of modules that call a given module
- Out-degree (Fan-out)
  - A count of the number of modules that are called by a given module
- Path
  - A sequence of consecutive (directed) edges, some of which may be traversed more than once during the sequence
- Simple path

Control-Flow Structure - 3

- Modules that have large fan-in or large fan-out may indicate a poor design. Such modules have probably not been decomposed correctly and are candidates for redesign

Cognitive Complexity

- Measures the effort required to understand the software

Cyclomatic Complexity

Cyclomatic Complexity - 1

- The measurement of cyclomatic complexity by McCabe was designed to indicate a program's testability and understandability (maintainability)
- It is a classical graph theory cyclomatic number, indicating the number of regions in a graph

Cyclomatic Complexity - 2

- As applied to software, it is the number of linearly independent paths comprising the program
- As such it can be used to indicate the effort required to test a program
- To determine the paths, the program procedure is represented as a strongly connected graph with a unique entry and exit point

Cyclomatic Complexity

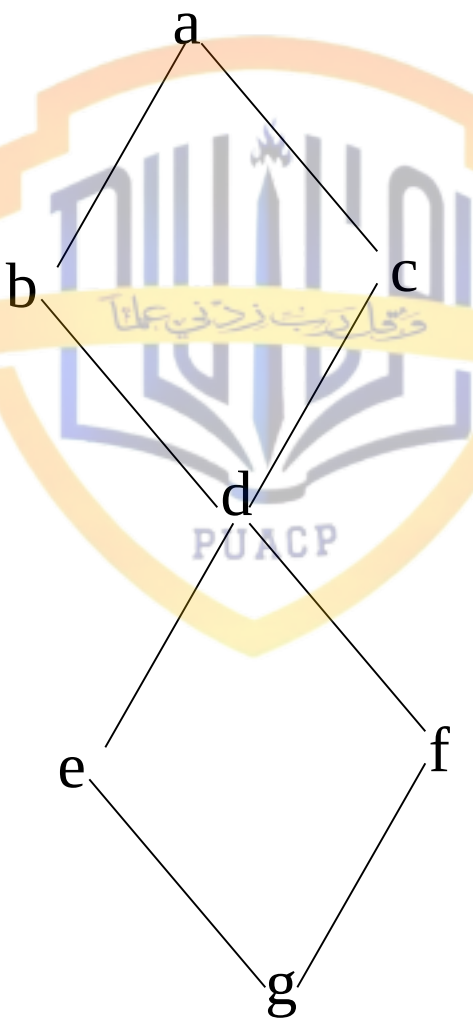
$$M = V(G) = e - n + 2$$

where

V(G) = cyclomatic number of G

e = number of edges
n = number of nodes

Simple Control Graph Example



Cyclomatic Complexity - 4

- What we have seen is a control graph of a simple program that might contain two if statements. If we count the edges, nodes, and disconnected part of the graph, we see that
  - e = 8, n = 7
  - and that M = 8 – 7 + 2 = 3
- Note that M is equal to number of binary decisions plus one (1)

Cyclomatic Complexity - 5

- The cyclomatic complexity metric is additive. The complexity of several graphs considered as a group is equal to the sum of the individual graphs' complexities
- However, it ignores the complexity of sequential statements
- The metric also does not distinguish between different kinds of control flow complexity such as loops versus IF-THEN-ELSE statements or cases verses nested IF-THEN-ELSE statements

Cyclomatic Complexity - 6

- To have good testability and maintainability, McCabe recommended that no program should exceed a cyclomatic complexity of 10. Because the complexity metric is based on decisions and branches, which is consistent with the logic pattern of design and programming, it appeals to software professionals

Cyclomatic Complexity - 7

- Many experts in software testing recommend use of the cyclomatic representation to ensure adequate test coverage; the use of McCabe's complexity measure has been gaining acceptance in practitioners

Essential Complexity - 1

- This term is similar to “cyclomatic complexity” with an important difference. Before calculating the essential complexity level the graph of program flow is analyzed and duplicate segments are removed. Thus, if a module or code segment is utilized several times in a program, its complexity would only be counted once rather than at each instance

Essential Complexity - 2

- Thus, essential complexity eliminates duplicate counts of reusable material and provides a firmer basis for secondary calculations such as how many test cases might be needed. This form of complexity is usually derived by tools which parse source code directly

Summary

References

- Metrics and Models in Software Quality Engineering by Stephan H. Kan (Chapter 4.1.4 and Chapter 10)
- Software Metrics: A Rigorous & Practical Approach, by Norman E. Fenton and Shari L. Pleeger, 2nd Edition, PWS Publishing Company, 1997 (Chapter 7 and Chapter 8)